
title: Build Work Estimate — Harbor Line Logistics -- dispatch agent author: Dewain Robinson
build_platform: claude-code target_platform: copilot-studio estimate_id: est_20260903T000000_sample1
generated: 2026-09-03T00:00:00Z

Build Work Estimate — Harbor Line Logistics -- dispatch agent

Author: Dewain Robinson · **Estimate id:** est_20260903T000000_sample1 · **Generated:** 2026-09-03T00:00:00Z

SAMPLE — BUILD ESTIMATE ONLY, NOT A QUOTE

This document is generated by a **sample estimator** published as a demonstration of *how* an organization could build one. It is not a quote, a bid, a budget authority, or a commitment of any kind.

It estimates the work of building only — never the cost of running what gets built. Runtime, end-user licences, infrastructure, and human labour are all outside its scope and are not represented in any figure below.

The figures are estimates and will be wrong. They are derived from historical consumption patterns that may not resemble the work being estimated. Observed cost for comparable work in the calibration data spans more than 100x between the cheapest and most expensive instances. Treat the range as the estimate; treat the point figure as the midpoint of a guess.

Before any real budgeting use, this estimator must be modified for your organization — recalibrated against your own usage history, repriced against your contracted rates rather than list price, and adjusted for your own delivery patterns, model choices, and review overhead.

Rates verified 2026-09-03 and change without notice. Calibration source: measured · Generated 2026-09-03T00:00:00Z

Estimate summary

What was estimated

INPUT	VALUE
Built with (development tool)	Claude Code — metered in USD (tokens)
Built on (target environment)	Microsoft Copilot Studio — metered in Copilot Credits
Built by (model)	claude-sonnet-5 80% / claude-opus-5 20% — per-turn cost repriced x0.7123
Target harness	github-copilot — bills for build, test and evaluation
Licensing	Seat — Claude Max, \$200.00/month x 1 seat x 1 month = \$200.00 over the build
Specification	draft — confidence in sizing: low
Evaluation cycles planned	4
Contingency reserve	25%
Calibration	Measured, 24 local sessions

Totals

	BUILD — USD	TARGET — COPILOT CREDITS	COMBINED (USD)
Low	\$164.31	28,418	\$448.49
Likely	\$634.11	32,290	\$957.01
High	\$3,726.17	40,035	\$4,126.52
Likely + 25% reserve	\$792.64	40,362	\$1,196.26

Plan for \$957.01. Hold \$1,196.26 including the 25% reserve.

Two meters, not two options. The build and target figures are spent on the same project over the same period and add together; they are not alternatives to choose between.

- Copilot Credits are converted at \$0.01 each so the two meters can be shown in one column. They are separate budgets drawn on different accounts.
- The build figure is **notional** — on seat licensing no additional money is invoiced. See Licensing below for the share of the seat this build actually consumes.

- The build range comes from observed spread in comparable work; the target range from running 3 to 6 evaluation cycles instead of 4.

Everything below explains how each of these figures was reached.

What this was sized from

	REFERENCE
Functional specification	Harbor Line dispatch requirements (illustrative)
Technical specification	Copilot Studio agent design note (illustrative)
Status	<code>draft</code> — Draft only. Sizes will move as it settles.
Confidence in sizing	<code>low</code>

The specification is a draft. Sizes will move as it settles, and a draft that grows is the most common reason a build overruns its estimate.

Was the breakdown challenged?

No research review is recorded. This estimate prices the breakdown as written. Nothing has checked whether it is complete, whether an item is plausibly sized, or whether the specification is detailed enough to size from at all.

Reviewing it is an available improvement, and the breakdown is the weakest input in the whole estimate.

Platforms

	PLATFORM	METERED IN
Built with	Claude Code	USD (tokens)
Built on	Microsoft Copilot Studio	Copilot Credits

These are different questions and different meters, and the same project spends on **both**. The building happens in a coding agent authoring the agent definition; the target platform is where it is deployed, previewed, evaluated and validated. Copilot Studio is a destination, not a build tool.

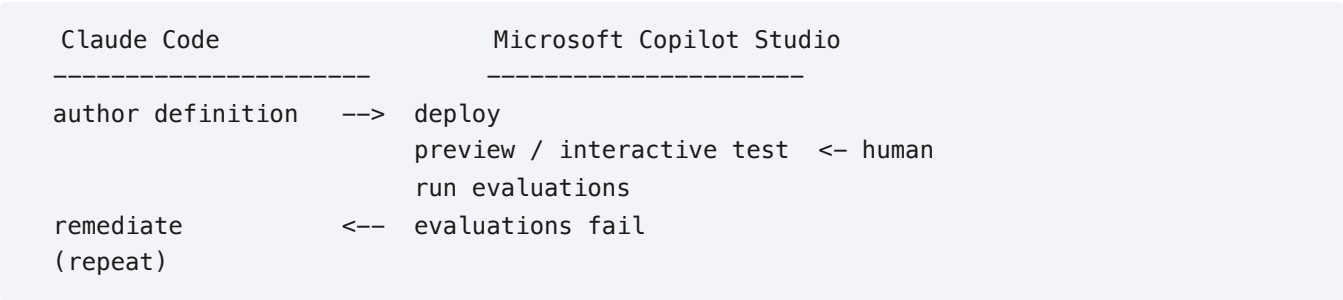
This is not a platform comparison tool. It reports one chosen build platform and one chosen target, each in its own meter. Platform decisions are not made on cost alone — capability, existing skills, governance, integration, and support all matter more than a build-time figure — and using this report to pick a platform would be using it for something it was not designed to answer.

Scope — this estimates the build, not the run

This figure covers the work of **building** the thing. It does not cover operating it afterwards.

OUT OF SCOPE	WHERE TO GO INSTEAD
Runtime / operational cost of what gets built	Copilot Studio agent usage estimator
End-user seat licences (M365 Copilot, Power Platform)	Your licensing/procurement process
Infrastructure, hosting, storage, egress	Your cloud cost tooling
Human labour — PM, design, QA time	Your delivery estimation process
Ongoing maintenance and support after delivery	A run cost, not a build cost

The build loop



This estimate plans **4 evaluation cycles**. Each cycle after the first adds **25%** of the original build back as remediation, giving a build-side multiplier of **1.75x**. An estimate that prices only the first pass is planning for a build where every evaluation passes first time.

Cost by component

Each component's share of **both** meters. The build column is the work of authoring it; the credits column is the evaluation volume it generates on the target platform.

No component declared its evaluation cases, so target credits are not attributed here — they appear only as a solution-wide total. Adding `eval_cases:` to each item shows which components actually drive the credit burn. They are not distributed by guesswork.

COMPONENT	SIZE	TURNS	BUILD
Agent instructions, topics and tool definitions	medium	1,218	\$319.38
Dataverse schema and rate-card importer	small	289	\$75.69
Legacy AS400 field mapping research	exploration	100	\$26.14
Driver mobile handoff screens	medium	812	\$212.91
Total		2,419	\$634.12

Environments

No environments were declared. This estimate prices the solution as if it were built once, into one place. A real delivery provisions dev, QA, test and production — each needing infrastructure applied, pipelines wired, configuration and secrets set, data seeded, and the agent deployed.

Declaring them is the single largest correction available to this estimate.

Build — Claude Code

	AMOUNT
Base estimate (incl. remediation)	\$634.11
Reserve (25%)	\$158.53
Budget ask	\$792.64
Observed low	\$164.31
Observed high	\$3,726.17

Reserve adequacy

The reserve does not cover observed variance.

A 25% reserve reaches \$792.64. Comparable work has reached \$3,726.17. Full coverage would require 488%.

ITEM	SIZE	FILES	TURNS	ESTIMATE	RANGE	N
Agent instructions, topics and tool definitions	medium (brownfield)	11	1,218	\$319.38	\$70.37 – \$1,632.03	5
Dataverse schema and rate-card importer	small	4	289	\$75.69	\$41.28 – \$297.36	6
Legacy AS400 field mapping research	exploration (brownfield)	0	100	\$26.14	\$5.74 – \$1,071.44	7
Driver mobile handoff screens	medium	8	812	\$212.91	\$46.92 – \$725.34	5

Calibration basis

Measured from 24 local sessions spanning 2026-06-01 to 2026-08-30.

Cost per agent turn: \$0.32, at list price — an organization on contracted rates must substitute its own.

Which model builds it

	MODEL
Declared for this build	claude-sonnet-5 80% / claude-opus-5 20%
Measured in the calibration profile	claude-opus-5 55% / claude-sonnet-5 45%

Repricing the measured \$0.32 per turn by x0.7123 gives \$0.23 — this model costs less per token than the mix the measurement was taken on.

How the ratio was derived

Each row is a measured dollar share of the real per-turn cost, times a ratio of published per-model rates. There is no free parameter.

COST COMPONENT	MEASURED SHARE	THIS MODEL \$/1M	CALIBRATION \$/1M	CONTRIBUTION
Cache read	66%	0.2600	0.3650	0.4701
Cache write	25%	3.2500	4.5625	0.1781
Output	9%	13.0000	18.2500	0.0641
Ratio				0.7123

What this does not capture. The rescale prices *tokens*, not capability. It does **not** model that a cheaper or weaker model may need **more turns** to do the same work. That effect is real and is not derivable from published rates, so it is named here rather than folded silently into the number. The rescale also holds the measured token profile — context size, output length, cache behaviour — constant.

Licensing

Seat-based licensing (Claude Max). Usage draws on an allowance already paid for, so no additional money changes hands for this build -- but the seat is not free.

Developer-months of allowance available	1 (1 seat x 1 month)
Share of that allowance	134%
Seat spend over the build	\$200.00
Attributable cost of this build	\$267.93
Already committed to other work	45%
Total committed	179%

The attributable cost is the seat's monthly price apportioned by the share of the allowance this build consumes. Nothing extra is invoiced, but this is the real cost of the capacity the build uses up.

Allowance overrun

This build plus existing workload exceeds the allowance by 79%.

Work will stall at the limit unless overage is enabled, at which point it bills on top of the seat. Overage exposure at the same rate: **\$157.93**.

Options: spread the build across allowance periods, move part of it to consumption billing, add seats, or reduce the other committed work.

Window risk

Seat allowances refill on **5-hour rolling and weekly** windows, not only monthly. A build that fits comfortably in a month can still exhaust a short window and stall.

This build is marked as concentrated — compressed into a short period. Monthly headroom will not protect it; expect to hit shorter windows and plan for pauses.

Notional value at list rates: **\$634.11** — what this build would cost on consumption billing. Shown for scale; it is not a charge.

Target platform — Microsoft Copilot Studio

Target harness: `github-copilot` — GitHub Copilot harness bills from the moment you start building. Creating a solution with natural language, previewing, testing, and generating evaluations all consume credits. Credits cover LLM tokens, tools (knowledge and MCPs), and the harness itself.

BUILD-AND-TEST ACTIVITY	CREDITS	AT \$0.01/CC	BASIS
Interactive validation in the Copilot Studio interface	16,800	\$168.00	16.0 human hours x 25 interactions/hour = 400 interactions
Evaluation runs	15,360	\$153.60	40 cases x 3 repeats x 4 cycles = 480 runs
Agent flow runs during build and test	130	\$1.30	1,000 actions across 4 cycles at 13 CC per 100 -- test runs in the designer and test chat are exempt
Total	32,290	\$322.90	
Reserve (25%)	8,072	\$80.73	required contingency
Budget ask	40,362	\$403.62	

Reasoning-model surcharge: 25,840 credits (\$258.40). Reasoning models bill the feature rate *plus* the premium token tier, so the effective tier is `premium` whatever the `standard` tier selected.

The evaluation loop

Evaluation is not a one-off gate. Microsoft's guidance is an explicit cycle — define tests, run evaluations, analyse results, improve the agent, repeat — with a target pass rate of 80–90% and near 100% on core tests.

This estimate plans **4 cycles**: 480 evaluation runs in total, plus the remediation each failed cycle sends back to the build platform.

Velocity, not just cost. Evaluations are capped at **20 per agent node per day**, so this volume needs a minimum of **24 days** of elapsed time however much budget is available.

Human validation — a dependency, not a cost line

16.0 hours of interactive validation are planned in the Copilot Studio interface. Those hours are collected to size the test volume above — they are **not** estimated as labour cost, consistent with this tool metering agent consumption rather than people.

Someone must go into the interface, confirm behaviour, and make configuration changes between cycles. That step gates the loop, and no amount of build budget removes it.

Not included

- Production runtime once the agent is live
- Agent flow test runs in the designer or test chat (documented exemption)
- Capacity pack sizing and overage enforcement
- End-user Microsoft 365 Copilot licence offsets
- Human hours for validation (collected to size test volume only)

For production runtime consumption use Microsoft's [agent usage estimator](#).

Rates verified 2026-09-03. Sources: [billing rates](#) · [harness billing](#) · [evaluation limits](#) · [iteration guidance](#)

What is measured, and what is judgment

This estimate mixes two kinds of input. They are not equally trustworthy, and the difference is not visible in the figures themselves.

Measured or sourced

INPUT	BASIS
Cost per agent turn, bucket turn medians, cache behaviour	Derived from 24 real local sessions
Every provider rate	Published, each carrying a source URL and a verification date
Evaluation volume	Arithmetic on declared test cases, repeats and cycles
Azure consumption	Figures supplied by you; nothing is bundled or inferred

Judgment, not measurement

These shape the result and **were never measured against anything**. They are stated here so a reader can see which parts of the number would move if someone measured them.

FACTOR	VALUE	WHAT IT DOES
Brownfield factor	1.50x	Multiplies turns for work in an existing codebase. Judgment. Never measured against paired greenfield and brownfield work.
Remediation share per cycle	25%	Each evaluation cycle after the first adds this share of the build back as rework. Judgment. Real remediation depends on what the evaluations actually find.
Unknowns range widening	+25% per unknown	Widens the upper bound of an item per declared unknown. Judgment. The declared unknown count is itself a subjective input.
Environment provisioning share	25% per extra environment	Cost of applying infrastructure and pipeline work into each environment beyond the first. Judgment. Override it with <code>environments.provisioning_share</code> if you have a real figure.
Evaluation cycle range	-1 / +2 cycles	Produces the low and high bounds on the target side. Judgment. Nobody knows how many cycles a build will need until it runs.
Correction shrinkage k	k = 3	Pulls recorded actuals toward 1.0 so one data point cannot swing later estimates. Judgment, but a deliberately conservative one: it only ever reduces the influence of thin data.

Nothing here is invented at report time. Every figure above is either measured, supplied by you, or one of the listed factors applied to those. Where the estimator cannot attribute a cost honestly — target credits with no declared evaluation cases, for instance — it says so rather than distributing the total to make the table look complete.

Provenance of every figure

Every money figure and every grouped number above has been checked against the estimator's own derivation ledger. Each one is either measured from session history, a published rate carrying a source URL and verification date, a value you declared in the manifest, or arithmetic on those.

Nothing in this report is asserted without a derivation. The check is mechanical and runs on every build; a figure the estimator cannot account for fails validation rather than being printed.

Matching is exact at the precision each figure is rendered to. There is **no tolerance window**, so a value one away from a recorded one fails.

What it does **not** verify: that a recorded value appears in the right place. A renderer showing a real figure under the wrong label would pass. That is a different defect needing a different check, and claiming otherwise would be the overstatement this section exists to avoid.

Known limits

1. **Build only.** Nothing here is the cost of running the agent once it is live.
2. **Human validation is a dependency, not a line item.** Hours are collected to size test volume, never estimated as labour.
3. **Turn counts are the weakest input,** and thin buckets are flagged above.
4. **Rates go stale.** Re-verify against the sources cited.
5. **This is a sample.** Modify it for your organization before budgeting use.

Close the loop

```
python record_actual.py est_20260903T000000_sample1 --sessions <session-id> [...]
```

SAMPLE — build estimate only, not a quote. Excludes runtime cost. Modify for your organization before budgeting use.